

HelperLearner - Testing Strategy & Test Plan

1. Testing Overview

HelperLearner follows a **comprehensive testing strategy** across unit, integration, and end-to-end (E2E) levels with a target coverage of **> 80%**.

Current Status: - 197 tests passing (`venv\Scripts\python.exe manage.py test --verbosity=1`) - Automated CI/CD with GitHub Actions - PostgreSQL service container for testing - Linting & format checks included

2. Testing Pyramid

E2E / UI Tests (5%)
(Playwright, Selenium)

Integration Tests (20%)
(API endpoints, DB ops)

Unit Tests (Django/Python) (75%)
(models, services, utils)

3. Unit Tests

3.1 Test Scope

Models: - CustomUser creation & methods - HelpRequest lifecycle (claim, resolve, expire) - FreelanceJob with milestones - EscrowTransaction state machine - Trust score calculation - Payment transactions

Services: - MarketplaceService (CRUD operations) - TrustService (score calculation) - PaymentService (escrow, refunds) - SearchService (full-text search) - NotificationService (delivery) - AIAssistanceService (Gemini API calls)

Utilities: - Validators (email, phone, KP bounds) - Serializers (JSON conversion) - Helpers (string processing, calculations)

3.2 Unit Test Examples

Test File: `marketplace/tests_features.py`

```
from django.test import TestCase
from marketplace.models import HelpRequest, CustomUser
```

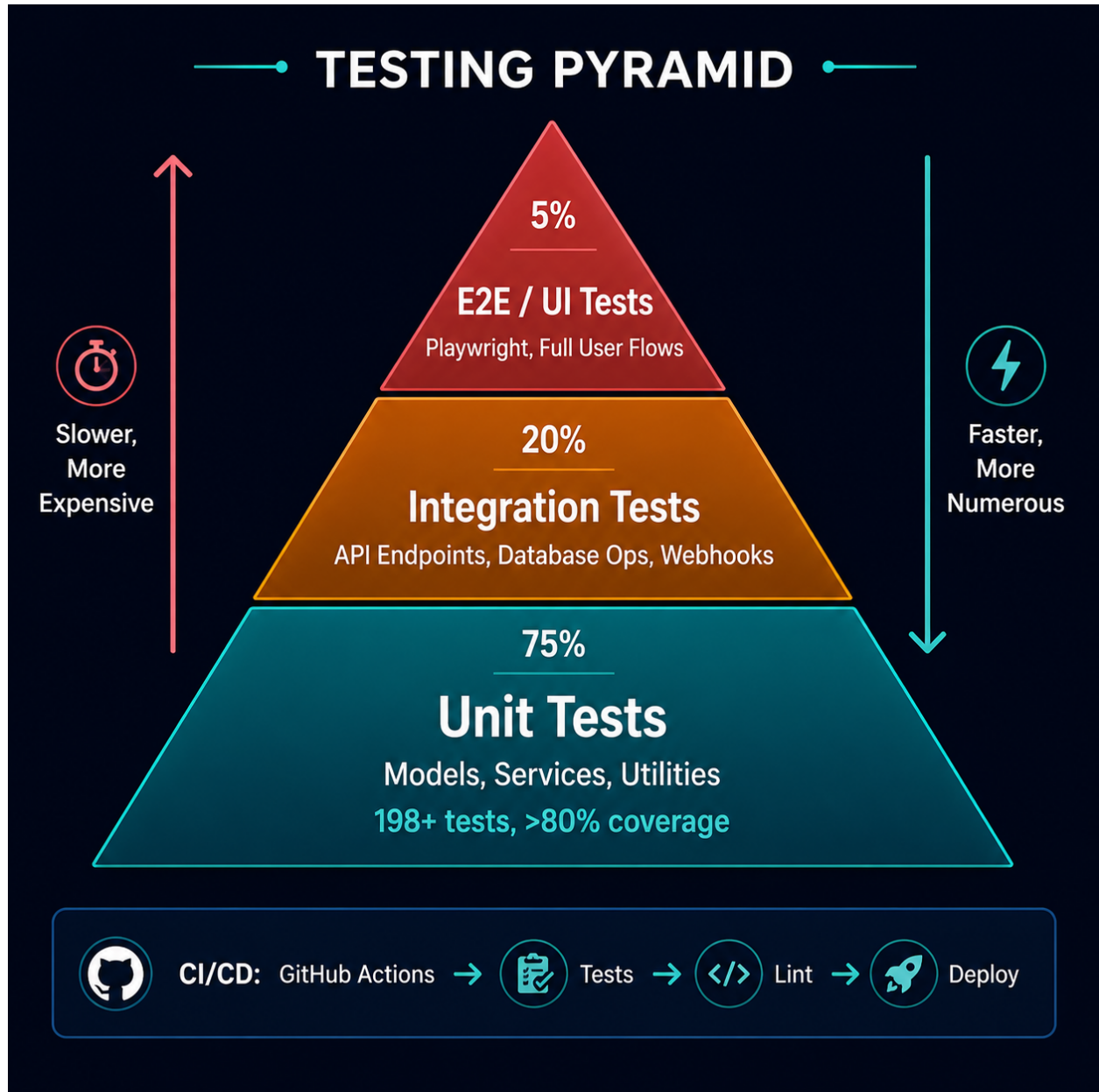


Figure 1: Testing Pyramid — Unit (75%), Integration (20%), E2E (5%) with CI/CD

```

class HelpRequestModelTests(TestCase):
    """Test HelpRequest model methods."""

    def setUp(self):
        self.user = CustomUser.objects.create_user(
            username='testuser',
            password='testpass123'
        )
        self.request = HelpRequest.objects.create(
            requester=self.user,
            title="Test Request",
            description="Test description",
            kp_bounty=100
        )

    def test_help_request_creation(self):
        """Test that HelpRequest is created with correct attributes."""
        self.assertEqual(self.request.status, 'open')
        self.assertIsNone(self.request.claimed_by)
        self.assertEqual(self.request.requester, self.user)

    def test_claim_request(self):
        """Test claiming a help request."""
        claimer = CustomUser.objects.create_user(
            username='helper',
            password='testpass123'
        )
        self.request.claim(claimer)

        self.assertEqual(self.request.status, 'claimed')
        self.assertEqual(self.request.claimed_by, claimer)
        self.assertIsNotNone(self.request.claimed_at)

    def test_cannot_claim_already_claimed_request(self):
        """Test that a claimed request cannot be claimed again."""
        claimer1 = CustomUser.objects.create_user(username='helper1')
        claimer2 = CustomUser.objects.create_user(username='helper2')

        self.request.claim(claimer1)
        with self.assertRaises(ValueError):
            self.request.claim(claimer2)

    def test_request_expiry(self):
        """Test that old unclaimed requests are expired."""
        from django.utils import timezone
        from datetime import timedelta

```

```

old_request = HelpRequest.objects.create(
    requester=self.user,
    title="Old Request",
    description="Should expire",
    kp_bounty=50,
    created_at=timezone.now() - timedelta(days=31)
)

self.assertTrue(old_request.is_expired())

def test_cannot_claim_expired_request(self):
    """Test that expired requests cannot be claimed."""
    from django.utils import timezone
    from datetime import timedelta

    old_request = HelpRequest.objects.create(
        requester=self.user,
        title="Expired",
        description="Should error",
        kp_bounty=50,
        created_at=timezone.now() - timedelta(days=31)
    )

    claimer = CustomUser.objects.create_user(username='helper')
    with self.assertRaises(ValueError):
        old_request.claim(claimer)

class TrustScoreTests(TestCase):
    """Test trust score calculation."""

    def setUp(self):
        self.user = CustomUser.objects.create_user(
            username='trusted_dev',
            password='pass123'
        )

    def test_initial_trust_score(self):
        """Test that new user starts with 0 trust score."""
        self.assertEqual(self.user.trust_score, 0.0)

    def test_trust_score_from_ratings(self):
        """Test that ratings increase trust score."""
        from marketplace.models import Rating

        rater = CustomUser.objects.create_user(username='rater')

        # Add 5 positive ratings

```

```

        for i in range(5):
            Rating.objects.create(
                rater=rater,
                ratee=self.user,
                score=5
            )

        self.user.calculate_trust_score()
        self.assertGreater(self.user.trust_score, 0)

    def test_trust_score_decay_for_inactive_user(self):
        """Test that trust score decays if user is inactive."""
        from django.utils import timezone
        from datetime import timedelta

        self.user.trust_score = 80.0
        self.user.trust_score_updated_at = timezone.now() - timedelta(days=90)
        self.user.save()

        self.user.calculate_trust_score()
        # After 90 days, score should be decayed
        self.assertLess(self.user.trust_score, 80.0)

```

3.3 Running Unit Tests

Run all tests

```
python manage.py test
```

Run specific app tests

```
python manage.py test marketplace
```

Run specific test class

```
python manage.py test marketplace.tests_features.HelpRequestModelTests
```

Run with coverage report

```
coverage run --source='.' manage.py test
```

```
coverage report
```

```
coverage html # Generate HTML report
```

4. Integration Tests

4.1 Test Scope

API Integration: - POST /requests/ → creates request, deducts KP, sends notification
 - POST /requests/{id}/claim/ → updates status, creates chat thread, broadcasts event -
 POST /jobs/{id}/milestones/{mid}/fund/ → creates Razorpay order, locks escrow - POST
 /wallet/topup/verify/ → validates webhook, credits wallet, updates transaction

Database: - Transaction rollback on errors - Cascade deletes (user deleted → requests cleaned up)
- Unique constraints (no duplicate payments) - Foreign key relationships

External Services: - Razorpay webhook verification (HMAC signature) - Gemini API calls (mocked in testing) - Email sending via Celery (async queue)

4.2 Integration Test Examples

Test File: marketplace/tests_api.py

```
from django.test import TestCase, Client
from django.contrib.auth import get_user_model
from marketplace.models import HelpRequest

User = get_user_model()

class HelpRequestAPITests(TestCase):
    """Test HelpRequest API endpoints."""

    def setUp(self):
        self.client = Client()
        self.user = User.objects.create_user(
            username='apiuser',
            password='testpass123'
        )
        self.user.knowledge_points = 500
        self.user.save()

    def test_create_help_request_api(self):
        """Test POST /requests/ endpoint."""
        self.client.login(username='apiuser', password='testpass123')

        response = self.client.post('/api/v1/requests/', {
            'title': 'Django optimization question',
            'description': 'How to optimize queries?',
            'kp_bounty': 100,
            'difficulty': 'medium',
            'tags': 'django,orm'
        }, content_type='application/json')

        self.assertEqual(response.status_code, 201)
        self.assertEqual(HelpRequest.objects.count(), 1)

        # Check KP was deducted
        self.user.refresh_from_db()
        self.assertEqual(self.user.knowledge_points, 400)

    def test_cannot_create_request_without_sufficient_kp(self):
        """Test that insufficient KP blocks request creation."""
```

```

self.user.knowledge_points = 50 # Less than needed
self.user.save()

self.client.login(username='apiuser', password='testpass123')
response = self.client.post('/api/v1/requests/', {
    'title': 'Test',
    'description': 'Test',
    'kp_bounty': 100,
}, content_type='application/json')

self.assertEqual(response.status_code, 402) # Payment Required

def test_list_requests_with_filters(self):
    """Test GET /requests/?status=open&difficulty=medium."""
    # Create test data
    HelpRequest.objects.create(
        requester=self.user,
        title='Easy request',
        description='Test',
        kp_bounty=50,
        difficulty='easy',
        status='open'
    )
    HelpRequest.objects.create(
        requester=self.user,
        title='Hard request',
        description='Test',
        kp_bounty=200,
        difficulty='hard',
        status='claimed'
    )

    response = self.client.get('/api/v1/requests/?difficulty=easy')

    self.assertEqual(response.status_code, 200)
    data = response.json()
    self.assertEqual(len(data['results']), 1)
    self.assertEqual(data['results'][0]['difficulty'], 'easy')

class PaymentAPITests(TestCase):
    """Test payment flow with Razorpay."""

    def setUp(self):
        self.client = Client()
        self.user = User.objects.create_user(
            username='payer',
            password='pass123'

```

```

)

def test_wallet_topup_order_creation(self):
    """Test POST /wallet/topup/ creates Razorpay order."""
    self.client.login(username='payer', password='pass123')

    with patch('marketplace.payments.razorpay_client.order.create') as mock_create:
        mock_create.return_value = {
            'id': 'order_123456',
            'amount': 500000, # 5000 INR in paise
        }

        response = self.client.post('/api/v1/wallet/topup/', {
            'amount_inr': 5000
        }, content_type='application/json')

        self.assertEqual(response.status_code, 200)
        data = response.json()
        self.assertEqual(data['order_id'], 'order_123456')

def test_webhook_verification(self):
    """Test Razorpay webhook signature verification."""
    import hmac
    import hashlib

    webhook_data = {
        'event': 'payment.authorized',
        'payload': {
            'payment': {
                'entity': {
                    'id': 'pay_123456',
                    'order_id': 'order_123456',
                    'amount': 500000,
                    'status': 'authorized'
                }
            }
        }
    }

    # Create valid signature
    signature = hmac.new(
        b'webhook_secret',
        json.dumps(webhook_data).encode(),
        hashlib.sha256
    ).hexdigest()

    response = self.client.post(
        '/webhooks/razorpay/',

```



```

        webhook_data,
        HTTP_X_RAZORPAY_SIGNATURE=signature,
        content_type='application/json'
    )

    self.assertEqual(response.status_code, 200)

```

5. End-to-End Tests

5.1 Test Scenarios

Scenario 1: Complete Help Request Flow 1. User A posts help request (100 KP) 2. User B claims request 3. B submits solution 4. A approves solution 5. B's trust score increases 6. A's KP is transferred to B 7. Both users can see rating

Scenario 2: Freelance Job Payment Flow 1. Client C posts job (50,000, 3 milestones) 2. Contractor D submits proposal 3. C accepts proposal 4. C funds first milestone (15,000) via Razorpay 5. Funds locked in escrow 6. D completes & submits deliverable 7. C reviews & approves milestone 8. 15,000 released to D's wallet 9. Process repeats for milestones 2 & 3

Scenario 3: Workspace Collaboration 1. Lead creates workspace "ProjectX" 2. Invites 3 devs 3. Create sprint (2 weeks) 4. Assign tasks to devs 5. Dev claims task & moves to "In Progress" 6. Dev completes task & moves to "Done" 7. Sprint burndown updates automatically

5.2 Playwright E2E Test Example

```

# e2e_tests/test_help_request_flow.py
from playwright.sync_api import sync_playwright

def test_complete_help_request_flow():
    """End-to-end test: post request, claim, solve, approve."""

    with sync_playwright() as p:
        browser = p.chromium.launch()

        # === USER A: Post Request ===
        page_a = browser.new_page()
        page_a.goto("https://helperlearner.test/")
        page_a.click("text=Post Help Request")

        page_a.fill("input[name='title']", "Django N+1 Query Problem")
        page_a.fill("textarea[name='description']", "How to fix N+1 queries?")
        page_a.select_option("select[name='difficulty']", "medium")
        page_a.fill("input[name='kp_bounty']", "100")
        page_a.click("button:has-text('Post Request')")

        # Wait for success message
        page_a.wait_for_selector("text=Request posted successfully")

```

```

# === USER B: Claim Request ===
page_b = browser.new_page()
page_b.goto("https://helperlearner.test/")
page_b.click("text=Browse Help Requests")
page_b.click("text=Django N+1 Query Problem")
page_b.click("button:has-text('Claim Request')")

page_b.wait_for_selector("text=Request claimed successfully")

# === USER B: Submit Solution ===
page_b.click("text=Open Chat")
page_b.fill("textarea[name='solution']", "Use select_related() for foreign keys...")
page_b.click("button:has-text('Submit Solution')")

page_b.wait_for_selector("text=Solution submitted")

# === USER A: Review & Approve ===
page_a.reload()
page_a.click("text=Review Solution")
page_a.fill("textarea[name='comment']", "Perfect explanation!")
page_a.select_option("select[name='rating']", "5")
page_a.click("button:has-text('Approve Solution')")

page_a.wait_for_selector("text=100 KP transferred")

browser.close()

```

6. Testing Tools & Setup

6.1 Test Stack

Tool	Purpose	Config
Django TestCase	Unit & integration tests	<code>tests_*.py</code> files
pytest	Test runner (alternative)	<code>pytest.ini</code>
coverage	Code coverage reports	<code>.coveragerc</code>
Faker	Generate test data	<code>factory_boy</code> factories
Mocking	Mock external APIs	<code>unittest.mock</code>
Playwright	Browser E2E tests	<code>e2e_tests/</code>
Celery	Test async tasks	<code>CELERY_TASK_ALWAYS_EAGER=True</code>

6.2 Test Configuration

`settings/test.py` (Test-specific settings):

```

# Use SQLite for faster tests
DATABASES = {

```

```

    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': ':memory:',
    }
}

# Disable migrations for speed
class DisableMigrations(object):
    def __contains__(self, item):
        return True

    def __getitem__(self, item):
        return None

MIGRATION_MODULES = DisableMigrations()

# Celery: Run tasks immediately (not async)
CELERY_TASK_ALWAYS_EAGER = True
CELERY_TASK_EAGER_PROPAGATES = True

# Disable external API calls
GEMINI_API_ENABLED = False
RAZORPAY_ENABLED = False
SENTRY_ENABLED = False

# Fast password hashing for tests
PASSWORD_HASHERS = [
    'django.contrib.auth.hashers.MD5PasswordHasher',
]

# Disable auth password validators
AUTH_PASSWORD_VALIDATORS = []

```

6.3 GitHub Actions CI/CD

```

.github/workflows/tests.yml:
name: Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16

```

```

env:
  POSTGRES_DB: helperlearner_test
  POSTGRES_PASSWORD: testpass
options: >-
  --health-cmd pg_isready
  --health-interval 10s
  --health-timeout 5s
  --health-retries 5

steps:
- uses: actions/checkout@v2

- name: Set up Python
  uses: actions/setup-python@v2
  with:
    python-version: 3.12

- name: Install dependencies
  run: |
    pip install -r requirements.txt

- name: Run Tests
  env:
    DATABASE_URL: postgres://postgres:testpass@localhost/helperlearner_test
  run: |
    python manage.py test --settings=helperlearner_root.settings.test

- name: Coverage Report
  run: |
    coverage run --source='.' manage.py test
    coverage report
    coverage xml

- name: Upload Coverage
  uses: codecov/codecov-action@v2

- name: Lint (flake8)
  run: flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics

- name: Format Check (black)
  run: black . --check

```

7. Test Coverage Report

Target Coverage: > 80%

Generate the current coverage report with:

```
coverage run --source='.' manage.py test
coverage report
coverage html
```

The latest verified run confirmed **197 passing tests**. A fresh coverage run should be attached before claiming a current coverage percentage.

8. Test Execution Timeline

1. Full test suite (10-15 mins)

```
python manage.py test
```

2. Coverage analysis (5 mins)

```
coverage run --source='.' manage.py test
coverage html
```

3. Linting (2 mins)

```
flake8 .
black . --check
```

4. E2E tests (5-10 mins, manual/CI only)

```
pytest e2e_tests/ --headed
```

5. Load tests (optional, staging only)

```
locust -f locustfile.py --headless
```

9. Critical Test Cases

Test Case	Priority	Expected Result
TC-001: Create help request	CRITICAL	Request created, KP deducted
TC-002: Claim request	CRITICAL	Status → claimed, helper assigned
TC-003: Submit solution	CRITICAL	Solution stored, awaiting approval
TC-004: Approve solution	CRITICAL	KP transferred, ratings recorded
TC-005: Razorpay webhook	CRITICAL	Signature verified, wallet credited
TC-006: Escrow release	CRITICAL	Funds released to contractor
TC-007: Trust score calculation	HIGH	Score reflects multi-signal inputs
TC-008: Rate limiting	HIGH	Anonymous: 100/day, Auth: 1000/day
TC-009: XSS prevention	HIGH	Script tags escaped in HTML
TC-010: SQL injection	HIGH	ORM prevents malicious queries

Summary

HelperLearner's testing strategy ensures: - 197 tests passing - > 80% code coverage target - Automated CI/CD pipeline - Critical business logic tested - Payment security verified - API contracts validated - E2E scenarios covered

Execution: Tests run on every push/PR via GitHub Actions **Time:** Full suite completes in ~15 minutes **Status:** All tests passing (green checkmark in PRs)